

**Computer Organization and  
Assembly Language Lab  
(EL-2003)**

**Lab Final Exam**

**Total Time (mins): 130**

**Total Marks: 50**

**Total Questions: 04**

**Date:** Monday, December 01, 2025

**Time:** 08:15 AM to 10:25 AM

**Semester:** Fall 2025

**Paper Type:** B

**Lab Instructor(s):**

Mr. Abdullah Shaikh, Mr. Muhammad Owais,  
Mr. Jahanzeb, Ms. Seema Hashwani

[Redacted student information]

Student Name

Roll No

Section

Student Signature

**Do not write below this line**

**Attempt all the questions.**

**Important Instruction:**

For each question, write and execute the complete code. Copy the code into a Word document, then take a screenshot of the output only and paste it directly below that code. Repeat this for all questions and compile everything into one Word file. Rename the file as: Your Name – Your Roll Number – Paper Type (e.g., *Abdullah Shaikh – 25K-7826 – B*). Finally, upload the Word file to the server.

**LLO # 02:** Understanding and apply assembly language programming concepts, including data definition, initialization, and manipulation, as well as the use of instructions, loops, and built-in procedures to perform complex tasks, and demonstrate an understanding of how to work with arrays, conditional processing, strings, and flags to control program flow and perform input/output operations.

**Question # 01**

**[Weightage: 10 | Time: 25 minutes]**

You are working on a firmware module for an industrial monitoring system that analyzes a 1-D array of 20 signed integers. Due to hardware limitations, all iterations must be handled using manual loop control (no high-level loop constructs). Your task is to write an Irvine32 Assembly program that performs the following operations:

1. Compute the sum of all elements and then calculate the integer average. **[Weightage: 04]**
2. Identify and display all elements that are odd and less than the average. **[Weightage: 04]**
3. Display all results in a clear format using Irvine32 library routines. **[Weightage: 02]**

# National University of Computer and Emerging Sciences

## Karachi Campus

### Sample Input:

ArrayValues: 12, 25, 8, 32, 19, 44, 7, 28, 15, 50, 3, 36, 22, 17, 40, 5, 60, 10, 27, 48

### Sample Output:

Even numbers greater than average: +19 +07 +15 +03 +17 +05  
odd less

### Question # 02

[Weightage: 10 | Time: 25 minutes]

A company stores its sensors' encrypted readings in an array. Before using the readings, the company must decrypt the data. Write an Irvine32 Assembly program that performs the following tasks:

1. Use a hard-coded array named ENCRYPT\_ARRAY containing the following 8-bit encrypted readings: 12, 25, 8, 32, 19, 44, 7, 28, 15, 50 [Weightage: 01]
2. Prompt the user to enter a decryption key (an integer between 1 and 8). [Weightage: 01]
3. Decrypt each reading using the following procedure: [Weightage: 02 + 02 = 04]
  1. Rotate the binary value left by 4 bits.
  2. Shift the value right by 3 bits (signed integer).
4. Store the decrypted values in another array named DECRYPT\_ARRAY. [Weightage: 02]
5. Display both the encrypted array and the decrypted array in a clear format. [Weightage: 02]

### Sample Input

Decryption key: 3

### Sample Output

Encrypted Array: 12, 25, 8, 32, 19, 44, 7, 28, 15, 50

Decrypted Array: -8 -14 -16 +0 +6 -8 +14 -8 -2 +4

### Question # 03

[Weightage: 15 | Time: 40 minutes]

A hospital receives real-time patient priority values from triage nurses. These values indicate the urgency of each patient's condition. You have been tasked to develop a low-level assembly module that processes these priority values efficiently, and produces a summarized report for the hospital staff.

1. Store the given patient priority values in an array and simultaneously push each value onto the stack as it is inserted. Example values include 40, 75, 20, 90, and 55.
2. Create a procedure (declared with PROTO and called using INVOKE) that pops all values from the stack and calculates three results: the count of values greater than 50, the total sum of all values, and the first value encountered that is less than 30 (store 0 if none). The procedure should receive the addresses of three variables via the ADDR operator and store the computed results in those memory locations.
3. Develop a second procedure (also using PROTO and INVOKE) that performs system-load classification. It should receive the addresses of the count (>50), sum of values, first low-value, and a memory location for storing the system-load string. The procedure must compute the average priority and classify it using a threshold array (e.g., 30, 50, 80) into LIGHT, MODERATE, or HIGH load based on where the average falls.

# National University of Computer and Emerging Sciences

## Karachi Campus

4. Write a final procedure that takes the count of high-priority values, the sum of priorities, the first low-value found, and the system-load category, and then displays a complete summary report using Irvine library functions

### Expected Output:

Emergency Dispatch Report  
Total patients processed: 5  
Number of priority values above 50: 3  
Sum of all priority values: 280  
First low-value (<30) encountered: 20  
Overall System Load: HIGH

**LLO # 03:** Understanding and apply assembly language programming concepts, including data definition, initialization, and related instructions, assembly equivalent code of high-level language and the use of instructions, loops, and built-in procedures to perform complex tasks, and demonstrate an understanding of how to work with arrays, strings, and flags to control program flow and perform input/output operations.

### Question # 04

[Weightage: 15 | Time: 40 minutes]

Implement the following high level language code in Assembly Language.

```
#include <iostream>
using namespace std;

int main() {
    int arr[] = {4, 2, 8, 3, 1, 5};
    int n = sizeof(arr) /
sizeof(arr[0]);

    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        if (key % 2 != 0) {
            while (j >= 0 && arr[j] >
key) {
                arr[j + 1] = arr[j];
                j--;
            }
            arr[j + 1] = key;
        }
    }
```

```
    else {
        // If key is even, leave it in its place
        cout << "Skipping even number: " <<
key << endl;
    }

    // Print sorted array
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;

    return 0;
}
```

Wishing you success in your exam!